Predictive Maintenance for Component X Failures in Heavy-Duty Scania Trucks: An Interpretable Machine Learning Approach

Umair Akram
Department of Computer Science,
Blekinge Institute of Technology (BTH),
Karlskrona, Sweden
mubu25@student.bth.se

***Abstract*—Deep learning models such as LSTMs are increasingly applied to industrial predictive maintenance due to their better performance. Nevertheless, their black-box nature makes them difficult to trust in safety-critical settings, where engineers and technicians need to understand why a prediction was made before acting on it. This paper investigates whether a classical, more interpretable machine learning approach can match or outperform a deep learning approach on a real-world truck failure prediction task. Using the Scania Component X dataset, we develop an XGBoost-based pipeline incorporating temporal feature engineering and SMOTE for imbalance handling. The model is compared against two LSTM baselines. XGBoost achieves an ROC–AUC of 0.734 and a recall of 0.648, outperforming the best LSTM model with an ROC–AUC of 0.651 and a recall of 0.373. Further analysis using SHAP reveals that the most influential features, such as cumulative sensor usage and recent temporal trends, have clear physical interpretations and thus make the model's predictions actionable for maintenance engineers. These findings suggest that, at least for datasets when failures are mainly driven by total accumulated usage, investing in careful feature engineering can substitute for, and even surpass, deep learning approaches, while keeping predictions transparent enough for engineers to act on.**

Keywords — Predictive Maintenance, Explainable AI, SHAP, Class Imbalance, Feature Engineering, LSTM

I. Introduction

In today's global world, the logistics and supply chain industry has become the backbone of the economy. Within this context, the importance of vehicle reliability has increased manyfold. When vehicles malfunction or break unexpectedly, it leads to operational downtime and unplanned, costly repairs. So, vehicle maintenance is an important aspect of the logistics industry. In order to ensure vehicle reliability, their faults need to be identified and solved beforehand so as to avoid shutdown or delays in the supply chain network.

Traditionally, the industry relied on reactive maintenance, that is, fixing the vehicle when a fault occurs. Nevertheless, with the exponential growth of Industrial Internet of Things (IIoT), Cyber-Physical Systems (CPS), and Artificial Intelligence (AI), the industry is transitioning from Reactive Maintenance to Preventive Maintenance (PdM) [1].

These techniques use real-time and past data to predict vehicle failures, plan maintenance more effectively, and reduce unexpected delays. This significantly improves productivity, lowers costs, and increases system reliability [2].

Recent research often involves Deep Learning (DL) architectures like Long Short-Term Memory (LSTMs) or Transformers, as they are quite effective with sequential data, such as time-series sensor readings without manual feature construction [3]. Nonetheless, this performance advantage comes at a cost: deep learning models operate as black boxes and thus reduce the level of trust among engineers [4].

A few studies have already applied machine learning to the Component X dataset as part of the IDA 2024 Industrial Challenge. Zhong and Wang [5] implemented CNN and LSTM architectures on the same data and found that deep learning models could identify failure patterns from raw sequences. Nevertheless, the interpretability of those models was not addressed. Carpentier et al. [6] explored classical models, including XGBoost with window-based feature extraction under a cost-sensitive evaluation framework. Our work differs from both as we explicitly compare interpretable ML against a deep learning baseline. Additionally, we use SHAP to explain predictions, which, to our knowledge, has not been done on this dataset in the IDA 2024 submissions.

This study investigates whether a classical, interpretable machine learning (ML) pipeline can match or exceed the performance of a deep learning (DL) model on this problem. The primary research question is: *Can classical algorithms effectively mine predictive failure patterns from complex sensor data to provide an interpretable alternative to deep learning for truck failure predictions?*

To answer this question, we apply an interpretable ML pipeline to the SCANIA Component X Dataset 25 [7]. We train XGBoost as the primary classifier, and compare both its performance and transparency against an LSTM trained on the same data. The LSTM is only for reference comparison. Our primary models are ML models.

Our results show that a gradient boosting model built on hand-crafted temporal features consistently outperforms a standard LSTM across all evaluated metrics. Additionally, it also provides interpretable insights through SHAP. This advantage comes from using features that capture the full history of each truck, whereas the LSTM relies only on a limited recent time window.

The paper is organized as follows: Section II describes the methodology following the CRISP-DM process model. Section III presents the results. Section IV discusses the findings. The limitations and conclusions are discussed in Sections V and VI, respectively.

II. Methodology

This study follows the CRISP-DM process model [8], which structures data mining projects into six phases: 1) business understanding, 2) data understanding, 3) data preparation, 4) modeling, 5) evaluation, and 6) deployment. The first five are addressed below, and deployment is beyond the scope of this study.

A. Business Understanding

The primary goal is to reduce the cost (financial and business) of component failures by ensuring planned maintenance. In this context, different types of errors have different impacts. A false negative (predicting a truck is healthy when it will fail) can lead to breakdowns, high repair costs, and operational delays. A false positive (flagging a healthy truck), on the other hand, only results in an

unnecessary inspection. Because missing a failure is more costly, we prioritize recall over precision.

To reflect this, we use the F2 score for threshold selection. The F2 score places more weight on recall than precision, indicating that detecting failures is more important than avoiding false alarms.

B. Data Understanding

The SCANIA Component X Dataset [7] is a real-world industrial dataset from Scania Trucks AB. It includes time-series sensor data, vehicle information, and failure labels.

The training set contains over 1.1 million observations from 23,550 trucks, while the test set includes about 198,000 observations from 5,045 trucks. Each observation has 105 sensor features, representing different types of measurements. Furthermore, each sensor variable is split into several bin columns. For example, sensor 167 is split into columns 167_0, 167_1, and so on. Each of these bins records how many time steps the truck spent within a corresponding value range during its operation.

The original dataset contains multi-class labels that indicate multiple failure types. We have combined these various failure types so as to convert the problem into a binary classification problem. The reason is that all the failure sub-types ultimately lead to the same repair action (often the replacement of Component X). So, distinguishing between them provides no operational benefit for our maintenance goal. That is why all non-zero labels are collapsed into a single failure class.

The dataset is highly imbalanced. About 9.65% of trucks in the training set have failures, compared to only 2.81% in the test set. This difference indicates a shift between training and test data and makes accuracy a poor metric. The reason is that predicting all trucks as healthy would still give high accuracy, but it would not give any useful results.

Missing values are rare (less than 1% per sensor) and appear to be due to occasional transmission gaps rather than systematic failures.

C. Data Preparation

- **Missing Values:** As mentioned earlier, missing values are not that abundant and are handled in three steps:
    - Missingness indicator columns are created for each sensor with missing entries. This preserves the signal that a reading was absent.
    - Then, apply forward fill within each truck's data, carrying the last observed value forward. This is appropriate for cumulative counters, where no new reading usually means no change in sensor value. The mean/median imputation was not considered because it ignores temporal order and thus is not a good strategy.
- **Preprocessing:** The sensor readings are cumulative counters. This means that they show total values of a given sensor over a particular instance of time rather than changes at each step. Since we are interested in how usage changes, we convert these into delta values using the *diff()* function. This is also important because tree-based models have no built-in sense of time order. So, a cumulative reading of 1,000 at step 2 and the same reading at step 50 would appear identical to the model, even though they indicate very different usage patterns.
- **Feature Engineering:** Classical tree-based classifiers require a fixed-width feature vector per sample. But each truck in our dataset has a variable-length time series. The key design challenge is to summarize this series in a way that captures the temporal dynamics relevant to failure prediction: long-term wear accumulation, medium-term behavioral trends, and short-term deviations that may indicate upcoming failure. To address this, each truck's time series is collapsed into a single feature row through ten groups of temporal statistics:
    - Global statistics (mean, standard deviation, maximum, summation) over the full history, capturing long-term wear.
    - Long-window statistics (mean, standard deviation) over the last 10 time steps.
    - Short-window statistics (mean, standard deviation) over the last 5 steps, together capturing recent trends at two granularities.

      The window sizes of 5 and 10 steps were selected to cover short operating-cycle fluctuations. Larger windows (20 and 30 steps) were tested and produced similar results at slightly higher computational cost. So, we opted for the smaller values.
    - The linear regression slope over the last 10 steps, encoding directional trend.
    - The deviation of the long-window mean from the global mean, indicating recent behavioral change.
    - A spike indicator, defined as the last observed value minus the long-window mean.
    - Global variability (minimum and range).
    - Acceleration, computed as the short-window minus long-window mean difference.
    - Global skewness.
    - The fraction of time steps where each sensor was missing.

  Truck specifications are one-hot encoded and appended. This produces *1,770* features in total.
- **Feature Selection:** The feature engineering solved the problem of cumulative counters, but it increased the complexity of dimensions as the number of features grew from about 100 features to 1770 features. Retaining all 1,770 features increases training time and the risk of noise overfitting. So, we need to select relevant features only, and for that, we apply a filter-based selection strategy using a lightweight XGBoost model (100 trees, max depth 4). It ranks feature

importances and retains only those features at or above the mean importance threshold. This reduces the feature space from 1,770 to 647 features (36.6% retained) without requiring cross-validated refitting, which would substantially increase computational cost. It is important to note that since the selector is trained on the same data as the final model, there is a mild optimistic bias. This means that the features that correlate with training labels by chance may be retained. A cross-validated selector would be more appropriate. But we did not investigate it due to computational complexity.

- **Imbalance Handling:** The 9.65% training failure rate is insufficient for reliable minority class learning. To handle this severe imbalance, we apply the Synthetic Minority Over-sampling Technique (SMOTE) because it generates new synthetic minority examples by interpolating between existing failure cases, rather than simply duplicating them. This exposes the model to a more diverse set of minority instances, and helps model generalize well. Simple random oversampling was rejected because duplication without interpolation does not increase the informational content of the training set. We use *sampling_strategy=0.3*, which raises the failure fraction from 9.65% to 23.1% (27,661 total samples after resampling). Figure 1 shows the class distribution before and after SMOTE.

  Full balance (50%) was tested, but it produced higher precision at the cost of noticeably lower recall and ROC-AUC. SMOTE is applied only to the training portion of a stratified 80/20 train-validation split. This ensures that the validation fold used for threshold selection contains only real observations and is not affected by synthetic examples.

Figure 1: Class distribution in the training set before (left) and after (right) SMOTE

D. Modeling

Three classifiers of increasing complexity are trained on the resampled feature set.

- *Logistic Regression* with *class_weight='balanced'* and standard scaling serves as a linear baseline.
- *Random Forest* (200 trees, max depth 20, *class_weight='balanced'*) provides a non- linear ensemble baseline that requires no hyperparameter tuning beyond tree count and depth.
- *XGBoost* as it performs well on tabular data and works well with SHAP for interpretability.

To compare interpretable ML models against deep learning model, two LSTM models are trained using PyTorch on the same dataset. Each model takes:

- A sequence of the last 100 time steps per truck.
- A set of overall features (total usage and number of time steps)

Each model uses two LSTM layers with a hidden size of 128 and a dropout rate of 0.3 between layers. They are trained with the Adam optimizer at a learning rate of $5e^{-4}$ and a batch size of 64. Two versions are tested:

- *Baseline LSTM*: uses a weighted loss to handle imbalanced data.
- Balanced LSTM: uses sampling to include more failure cases during training.

Early stopping is applied to both models to prevent overfitting. For all the ML models, the final classification threshold is chosen by maximizing the F2 score on a validation split and then applied to the test set. All three ML classifiers are therefore evaluated at their respective optimal thresholds rather than the default 0.5.

E. Evaluation

Model performance is assessed on four metrics, and all are reported at the F2-optimal threshold identified on the held-out validation split rather than the default 0.5:

- Recall (primary)
- Precision
- F1-Score
- ROC-AUC

Recall is prioritized due to the high cost of missed failures. Precision reflects the rate of false alarms. ROC-AUC is included as a threshold-independent measure for comparing models. Accuracy is not reported, as it is misleading under the strong class imbalance.

For explainability, SHAP TreeExplainer is applied to the final XGBoost model to compute feature contributions for all test samples. SHAP is chosen because it provides consistent and reliable explanations, unlike simpler methods such as permutation importance or LIME. It also offers both global feature importance and local, per-prediction explanations within a single framework.

III. RESULTS AND ANALYSIS

A. ML Model Performance

Table I summarizes the test set performance of all three ML classifiers after F2-optimal threshold selection, and Figure 2 highlights the ROC curves for three ML models.

Table I: ML model performance on the test set

Model	Recall	Precision	F1	ROC-AUC
Logistic Regression	0.303	0.057	0.097	0.626
Random Forest	0.669	0.049	0.091	0.702
XGBoost	0.648	0.058	0.106	0.734

XGBoost achieves the best ROC-AUC (0.734) and F1 (0.106). The substantial gap between Logistic Regression (ROC-AUC 0.626) and the two ensemble methods (with ROC-AUC of 0.734 and 0.702) confirms that the failure patterns are strongly non-linear. Furthermore, the manually engineered features contain meaningful signals that a linear classifier cannot fully exploit.

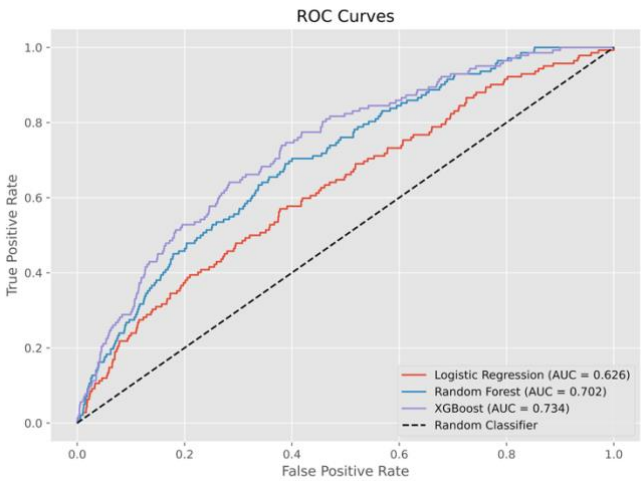

Figure 2: ROC curves for the three ML classifiers on the test set

Random Forest achieves the highest raw recall (0.669) but at a lower ROC-AUC (0.702), indicating that its recall advantage is threshold- driven rather than reflective of better overall discrimination. XGBoost's higher value on both the ROC-AUC and F1 makes it the preferred model.

We also evaluated these models on optimal F2 thresholds, and Figure 3 shows the confusion matrices for all three models. The F2-optimal thresholds, 0.256 for XGBoost and 0.229 for Random Forest, are well below the default 0.5.

For Logistic Regression, on the other hand, the optimal threshold fell at 0.510. This is essentially at the default boundary, and it indicates that the model's probability scores are already reasonably calibrated even with the class imbalance.

This further confirms that threshold tuning is essential on this dataset. The low precision across all models (below 0.06) is a direct consequence of the 2.81% failure rate in the test set. So, even a model with good discrimination will produce many false positives when the prior probability of failure is this low.

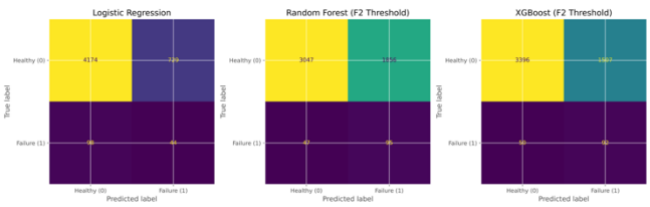

Figure 3: Confusion matrices for all three models after F2 threshold

B. Comparison with Deep Learning

Table II summarizes the test set performance of both LSTM models.

Table II: LSTM model performance on the test set

Model	Recall	Precision	F1	ROC-AUC
LSTM Baseline	0.373	0.054	0.094	0.651
LSTM Balanced	0.268	0.041	0.071	0.561

Both LSTM models underperform XGBoost on every metric, with the best LSTM achieving ROC-AUC 0.651 versus 0.734 for XGBoost, a gap of 0.083. The recall gap is larger: 0.373 versus 0.648. The Balanced LSTM performs considerably worse than the Baseline, suggesting that *WeightedRandomSampler* did not improve generalization on this dataset. This is probably because failure and healthy sequences have similar short-term sensor patterns that are difficult to separate with a fixed 100-step window. As shown in Fig. 4, both models triggered early stopping before epoch 15, with validation loss stagnating rather than improving. This indicates that the LSTM reached its capacity under the given data and architecture, and that deeper or longer-context models would be needed to narrow the gap with XGBoost.

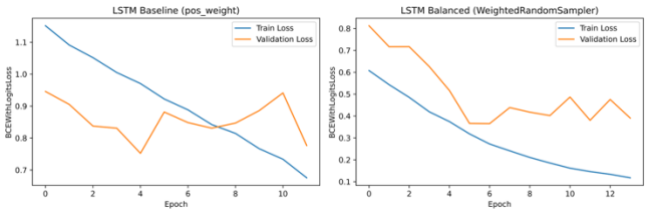

Figure 4: Training and validation loss curves for both LSTM models

C. Explainability via SHAP

Table III lists the five most important features for the XGBoost model by mean absolute SHAP value, computed over all 5,045 test predictions.

Table III: Top 5 features by mean absolute SHAP value

Feature	Mean (SHAP)	Interpretation
167_1_sum_global	0.183	Cumulative lifetime usage, sensor 167 bin 1
459_19_mean_w5	0.153	5-step recent average, sensor 459 bin 19
Spec_7_Cat1	0.124	Vehicle specification type 7
459_19_mean_w10	0.120	10-step recent average, sensor 459 bin 19
Spec_3_Cat1	0.114	Vehicle specification type 3

The dominant predictor, *167_1_sum_global*, measures the total accumulated activity of a specific sensor channel over the truck's full operational lifetime. This indicates that higher overall usage of any given component leads to higher wear-and-tear, and this indeed increases the chances of failure. The presence of both the 5-step and 10-step window averages of sensor 459 at positions 2 and 4 indicates that the model looks at both short-term changes and slightly longer trends in equipment working. Vehicle specification features (Spec_7, Spec_3) are also important because they indicate that different truck types may have different risks of failure.

The beeswarm plot (Figure 5) shows that trucks with high total usage (167_1_sum_global) tend to have higher SHAP values, pushing the model toward predicting failure. This makes sense, as more usage naturally increases wear. For sensor 459, the spread of SHAP values is roughly symmetric around zero. This indicates that both the unusually high and unusually low recent readings push the model toward a failure prediction. This finding is quite understandable because unusually high readings indicate heavy usage causing wear, or unusually low readings indicate something is not working properly.

For individual-level explanations, we also plotted the waterfall plot (Figure 6) that decomposes each prediction into per-feature contributions. For example, a truck predicted most confidently as failing (probability 0.951) was flagged mainly due to high cumulative usage and a recent spike. In contrast, the most confidently healthy truck (probability 0.0003) had all features pointing toward low risk.

These explanations help maintenance engineers see not just which trucks are at risk, but also which sensors are driving the prediction, allowing for more focused inspections.

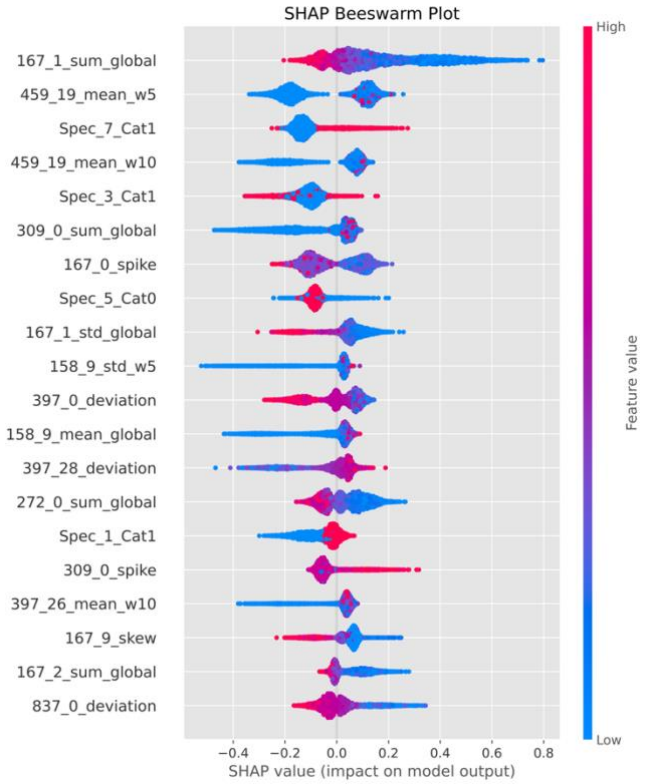

Figure 5: SHAP beeswarm plot for the XGBoost model (top 20 features)

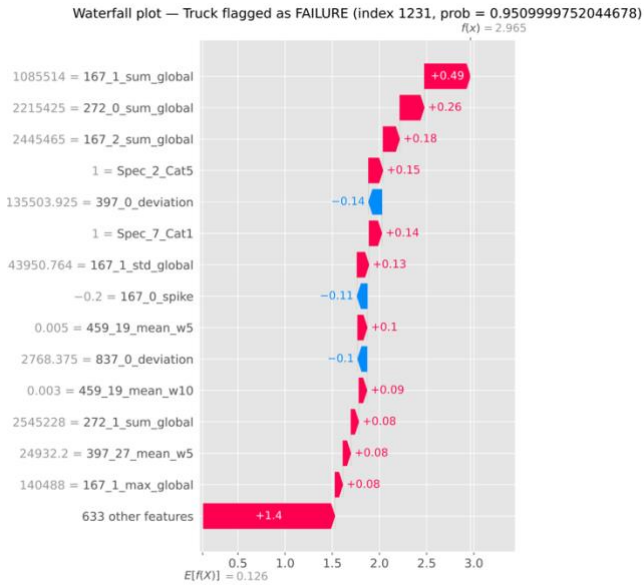

Figure 6: SHAP waterfall plot for the highest-confidence true positive

IV. DISCUSSION

The primary finding of this study is that a carefully engineered gradient boosting model outperforms an LSTM on every evaluation metric for this task.

In terms of performance, XGBoost achieves a ROC-AUC of 0.734 and a recall of 0.648, compared to 0.651 and 0.373 for the LSTM. The primary reason is a difference in the information available to each model. The feature engineering pipeline summarises a truck's full operational history into 647 interpretable statistics that cover multiple time scales, from the entire lifetime down to the most recent steps. The LSTM, on the other hand, processes only the last 100 time steps as a raw sequence, with aggregate features appended to partially compensate.

Additionally, for this problem, where cumulative wear is the main predictor, global lifetime features are more informative than short-term sequences.

From an interpretability perspective, SHAP analysis confirms that the top features make physical sense. That is, the cumulative usage, recent spikes, and vehicle type drive predictions. This makes the model more trustworthy than one with similar accuracy but opaque reasoning.

It is important to stress that one practical limitation is the low precision (0.058). That is, most failure alerts are false positives. This reflects the rarity of failures (2.81% in the test set) rather than a flaw in the model. In heavy-duty trucking, the cost of missing a breakdown is much higher than a false alarm. So even with low precision, the model can be valuable if thresholds are properly calibrated for real-world deployment.

This discrepancy is also caused by the difference between training and test data. As the model was trained with a higher failure rate (9.65%), but tested on a lower one (2.81%). Because of this, it tends to predict more failures, leading to more false positives. Adjusting the decision threshold for the test distribution could improve precision without greatly reducing recall.

V. LIMITATIONS

It is important to note that several limitations should be considered when interpreting the results of this study.

The most noteworthy is low precision (0.058), which means the system generates many false alarms. This is partly because failures are rare and sensor patterns of failing trucks overlap with healthy ones. Better thresholding using cost-aware analysis could improve this.

The feature selection process introduces a mild optimistic bias. Since the selector XGBoost was trained on the same data as the final model, it is highly possible that it retains noisy features. To address this, a cross-validation strategy could provide a more conservative feature set.

The training-test mismatch (9.65% vs. 2.81% failures) may also contribute to low precision, and thus reflects differences in time periods or conditions of truck operation.

Finally, the deep learning comparison is limited to a 2-layer LSTM with a 100-step window. More advanced architectures (deeper LSTMs, attention, Transformers) could perform better. Thus, the results show that XGBoost outperforms a straightforward LSTM, but do not rule out all deep learning approaches.

VI. CONCLUSIONS

This study aimed to determine whether the classical machine learning approach can effectively mine predictive failure patterns from complex sensor data and provide both a more accurate and more interpretable alternative to deep learning. The answer, based on the results presented here, is yes.

XGBoost achieves an ROC-AUC of 0.734 and a recall of 0.648, compared to 0.651 and 0.373 for the best LSTM model. The key reason is that explicit temporal feature engineering gives the ML model access to the full sensor history of each truck. The LSTM, on the other hand, only sees the last 100 steps. This shows that careful feature design can substitute for, and in this case exceed, the sequential pattern recognition ability of LSTMs.

The interpretability advantage is equally important. SHAP analysis identifies that the cumulative sensor usage and recent trends are the dominant predictors of failure. These are the features with direct physical meaning that a maintenance engineer can verify and act on. This is precisely what black-box deep learning cannot offer, and it is the core motivation for choosing an interpretable approach in a safety-critical industrial setting.

Future work should investigate longer LSTM sequence windows and Transformer architectures that process the full vehicle history. Cross-validated feature selection would also reduce the selection bias identified as a limitation.

VII. REFERENCES

[1] M. S. Azari, F. Flammini and San, "A systematic literature review on transfer learning for predictive maintenance in industry 4.0," *IEEE access,* vol. 11, pp. 12887--12910, 30 6 2023.

[2] T. P. Carvalho, F. A. Soares, R. Vita, R. d. P. Francisco, J. P. Basto and S. G. Alcalá, "A systematic literature review of machine learning methods applied to predictive maintenance," *Computers & Industrial Engineering,* vol. 137, p. 106024, 2019.

[3] T. Nguyen-Da, P. Nguyen-Thanh and M.-Y. Cho, "Real-time AIoT anomaly detection for industrial diesel generator based an efficient deep learning CNN-LSTM in industry 4.0," *Internet of Things,* vol. 27, p. 101280, 2024.

[4] D. Guidotti, L. Pandolfo and L. Pulina, "A Systematic Literature Review of Supervised Machine Learning Techniques for Predictive Maintenance in Industry 4.0," *IEEE Access,* 10 6 2025.

[5] J. Zhong and Z. Wang, "Implementing Deep Learning Models for Imminent Component X Failures Prediction in Heavy-Duty Scania Trucks," *International Symposium on Intelligent Data Analysis,* pp. 268--276, 2024.

[6] L. Carpentier, A. De Temmerman and M. Verbeke, "Towards Contextual, Cost-Efficient Predictive Maintenance in Heavy-Duty Trucks," *International Symposium on Intelligent Data Analysis,* pp. 260--267, 2024.

[7] T. Lindgren, O. Steinert, O. A. Reyna, Z. Kharazian and S. Magnússon, "SCANIA Component X Dataset: A Real-World Multivariate Time Series Dataset for Predictive Maintenance," 2024. [Online]. Available: https://researchdata.se/en/catalogue/dataset/2024-34/2. [Accessed 10 02 2026].

[8] R. Wirth and J. Hipp, "CRISP-DM: Towards a standard process model for data mining," in *Proceedings of the 4th international conference on the practical applications of knowledge discovery and data mining*, 2000, pp. 29--39.